

Linear Neural Networks for Regression

Dive Into DeepLearning Series

Hoai Linh DAO

25 November 2025

1. Basics

Regression problems pop up whenever we want to predict a numerical value. And Linear regression assumes that the expected value of the *target* y depends approximately linearly on the *feature* (or *covariate*) vector $\mathbf{x}$:

$$\mathbb{E}[y \mid \mathbf{X} = \mathbf{x}] \approx \mathbf{w}^\top \mathbf{x} + b,$$

where $\mathbf{w}$ denotes the weight vector and b is the bias term. The symbol $\approx$ indicates that the relationship between the *features* and the *target* in real-world data is not perfectly linear, but only approximated by the linear model.

Formally, we are given a dataset with n samples and d features, represented by the design matrix

$$\mathbf{X} \in \mathbb{R}^{n \times d},$$

together with a target vector

$$\mathbf{y} \in \mathbb{R}^n.$$

The linear regression model predicts all outputs as

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b, \quad \hat{\mathbf{y}} \in \mathbb{R}^n, \quad \mathbf{w} \in \mathbb{R}^d.$$

The learning objective is to find parameters $\mathbf{w}$ and b such that the predictions $\hat{\mathbf{y}}$ match the true targets $\mathbf{y}$ as closely as possible.

2. Loss Function

With this objective in mind, our task is to identify a pair $(\mathbf{w}, b)$ that performs best. To achieve this, we must solve an optimization problem. But what exactly are we optimizing?

Intuitively, the quality of any choice of $(\mathbf{w}, b)$ should reflect how close the model's predictions are to the true targets. In other words, we need a way to quantify the discrepancy between the predicted values and the actual values. A function that measures this discrepancy is called a *loss function*.

As a first and most straightforward choice, we consider the mean squared error (MSE). Any function that captures the distance between $\hat{y}$ and y could in principle serve as a loss, and many such loss functions exist. For now, however, we introduce only this specific one.

For a single training example $(\mathbf{x}^{(i)}, y^{(i)})$, the squared loss is defined as

$$\ell^{(i)}(\mathbf{w}, b) = \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2 = \frac{1}{2} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)^2.$$

Given a dataset of n examples, the empirical loss (or empirical risk) is the average loss across all samples:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \ell^{(i)}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)^2.$$

Training the model amounts to finding the parameters that minimize this total loss:

$$\mathbf{w}^*, b^* = \arg \min_{\mathbf{w}, b} L(\mathbf{w}, b).$$

3. Analytic Solution

We are given:

$$n = \text{number of samples}, \quad d = \text{number of features}.$$

To incorporate the bias term into the matrix form, we augment each feature vector with a constant 1. Thus the data matrix is:

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_d^{(1)} & 1 \\ x_1^{(2)} & x_2^{(2)} & \dots & x_d^{(2)} & 1 \\ \vdots & \vdots & & \vdots & \vdots \\ x_1^{(n)} & x_2^{(n)} & \dots & x_d^{(n)} & 1 \end{bmatrix} \in \mathbb{R}^{n \times (d+1)}.$$

Weight Vector and Labels:

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \\ b \end{bmatrix} \in \mathbb{R}^{d+1}, \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix} \in \mathbb{R}^n.$$

Prediction Vector:

$$\hat{y} = Xw, \quad \hat{y} \in \mathbb{R}^n.$$

We have,

Loss of one sample:

$$\ell^{(i)}(w) = \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2.$$

So, total MSE Loss:

$$L(w) = \frac{1}{n} \sum_{i=1}^n \ell^{(i)}(w) = \frac{1}{2n} \sum_{i=1}^n (Xw - y)_i^2.$$

and in vector form:

$$L(w) = \frac{1}{2n} \|Xw - y\|_2^2.$$

We expand the expression:

$$\begin{aligned} L(w) &= \frac{1}{2n} (Xw - y)^T (Xw - y) \\ &= \frac{1}{2n} (w^T X^T Xw - 2y^T Xw + y^T y). \end{aligned}$$

Applying Matrix Deviation:

$$\nabla_w (w^T X^T Xw - 2y^T Xw) = (X^T X + XX^T)w - 2X^T y = 2X^T Xw - 2X^T y.$$

$$\nabla_w L(w) = \frac{1}{2n} (2X^T Xw - 2X^T y) = \frac{1}{n} (X^T Xw - X^T y).$$

Optimize Loss Function, meaning set gradient = 0:

$$\nabla_w L(w) = 0 \iff X^T Xw = X^T y.$$

If the matrix $X^T X$ is invertible (i.e., X has full column rank, in other words, the columns of X are linearly independent), then:

$$\begin{aligned} (X^T X)^{-1} X^T X w^* &= (X^T X)^{-1} X^T y \\ I w^* &= (X^T X)^{-1} X^T y \\ w^* &= (X^T X)^{-1} X^T y. \end{aligned}$$

4. Optimizer

Although the closed-form solution

$$w^* = (X^\top X)^{-1} X^\top y$$

is elegant, solving it in practice is far from trivial. Real-world datasets typically contain correlated or redundant features, meaning that we do not control whether the columns of X are linearly independent. As a result, the matrix $X^\top X$ may be ill-conditioned or even non-invertible. Moreover, for high-dimensional data, computing this inverse explicitly becomes computationally expensive.

This motivates an alternative and more flexible approach: *gradient descent*.

Instead of solving for the optimum in a single analytic step, gradient descent iteratively updates the weights in the direction of decreasing loss:

$$w \leftarrow w - \eta \nabla_w L(w),$$

where η is the learning rate.

In practice, these updates eventually lead to a workable solution, though not necessarily the global minimum. The algorithm may converge to a local minimum or a saddle point, and the quality of the final solution depends heavily on the chosen optimization strategy.

The earliest form, *full-batch gradient descent*, computes the gradient using the entire dataset at once. While conceptually simple, it suffers from two major issues:

- Loading the full dataset into RAM or VRAM can be impossible when the dataset is large.
- Full-batch updates are completely deterministic. When the optimizer encounters a saddle point or a flat region, additional epochs (even hundreds of them) cannot help it escape.

At the opposite extreme, *stochastic gradient descent* updates the model after each individual training sample. The inherent noise in its updates often helps the optimizer escape saddle points. However, SGD introduces other limitations:

- Each update requires loading a single sample, creating significant data transfer overhead.
- Operations such as batch normalization perform poorly because they require statistics from multiple samples.

To balance the trade-offs between the two extremes, modern deep learning almost universally adopts *mini-batch SGD*. The dataset is split into smaller batches of a user-chosen size (e.g., 32, 64, 128), and each update uses only the samples in the current batch:

$$w \leftarrow w - \eta \nabla_w L_{\mathcal{B}}(w),$$

where

$$L_{\mathcal{B}}(w) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell^{(i)}(w).$$

Mini-batch SGD offers several advantages:

- It introduces moderate noise that helps avoid saddle points, similar to SGD.
- It reduces memory requirements compared to full-batch methods.
- It is compatible with modern techniques such as batch normalization.

5. The Meaning of the Gaussian Distribution

If you have ever wondered why the mean squared error (MSE) loss arises so naturally in linear regression, this section explains its origin. The key idea is that MSE is not chosen arbitrarily; rather, it follows directly from a probabilistic assumption about the noise present in real-world data.

Earlier we stated that the relationship between the features and the target is only approximately linear. To make this explicit, we model the unavoidable imperfections by introducing a random noise term:

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon.$$

Here, the equality sign is intentional: the term ϵ accounts for the intrinsic noise that accompanies real observations.

To proceed, we must specify a probability distribution for the noise ϵ . The most common and mathematically convenient assumption is that the noise follows a Gaussian distribution:

$$\epsilon \sim \mathcal{N}(0, \sigma^2).$$

A fundamental property of the Gaussian distribution is that any affine transformation of a Gaussian random variable remains Gaussian. Therefore,

$$\epsilon \sim \mathcal{N}(0, \sigma^2) \implies \mathbf{w}^\top \mathbf{x} + b + \epsilon \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x} + b, \sigma^2).$$

This means that, conditioned on $\mathbf{x}$, the target y is distributed according to

$$P(y | \mathbf{x}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2} (y - \mathbf{w}^\top \mathbf{x} - b)^2\right).$$

Because the samples $(\mathbf{x}^{(i)}, y^{(i)})$ are assumed to be independent, the likelihood of the entire dataset is the product

$$P(\mathbf{y} | \mathbf{X}) = \prod_{i=1}^n P(y^{(i)} | \mathbf{x}^{(i)}).$$

According to the principle of maximum likelihood, we seek the parameters w and b that maximize this probability. In practice, it is more convenient to minimize the negative log-likelihood:

$$-\log P(\mathbf{y} | \mathbf{X}) = \sum_{i=1}^n \left[\frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b)^2 \right].$$

Because of the constant σ , the only term influenced by the model parameters is

$$\sum_{i=1}^n (y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b)^2.$$

This is exactly the mean squared error loss. Thus, the familiar MSE objective emerges naturally from the assumption that the noise in the data is Gaussian.

More generally, different assumptions about the noise distribution lead to different loss functions. In this sense, the choice of loss function reflects our assumptions about how the data are generated.

6. Exercises

Exercise 1

1. We first rewrite the problem in matrix-vector form. Define

$$\mathbf{y} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n, \quad \mathbf{X} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} \in \mathbb{R}^{n \times 1},$$

and regard b as a one-dimensional parameter

$$w = b \in \mathbb{R}.$$

Our model predicts

$$\hat{\mathbf{y}} = \mathbf{X}w = b \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}.$$

The squared loss over the dataset is

$$L(w) = \frac{1}{2n} \|Xw - y\|_2^2 = \frac{1}{2n} (Xw - y)^\top (Xw - y).$$

Similar to Section 3, at the optimum we must have $\nabla_w L(w) = 0$, hence

$$X^\top X w = X^\top y.$$

In our special case,

$$X^\top X = \begin{bmatrix} 1 & 1 & \dots & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} = n, \quad X^\top y = \begin{bmatrix} 1 & 1 & \dots & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \sum_{i=1}^n x_i.$$

Therefore

$$nw^* = \sum_{i=1}^n x_i \implies w^* = b^* = \frac{1}{n} \sum_{i=1}^n x_i.$$

2. This problem is directly related to the normal distribution. If we assume that each observation is generated as

$$x_i = b + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

then the conditional density of x_i is

$$p(x_i | b) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x_i - b)^2\right).$$

Because the samples are independent, the likelihood of the dataset is

$$p(y | b) \propto \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - b)^2\right).$$

Maximizing this likelihood is therefore equivalent to minimizing

$$\sum_{i=1}^n (x_i - b)^2.$$

Thus, this least-squares problem is exactly the maximum likelihood estimation problem for the mean of a Gaussian distribution, and the solution

$$b^* = \frac{1}{n} \sum_{i=1}^n x_i$$

is the MLE of the Gaussian mean.

3. With the ℓ_1 loss

$$L(b) = \sum_{i=1}^n |x_i - b| = \|\mathbf{x} - b\mathbf{1}\|_1,$$

we work with subgradients because $|x_i - b|$ is not differentiable at $b = x_i$. For each term,

$$\partial|x_i - b| = \begin{cases} -1, & x_i > b, \\ 1, & x_i < b, \\ [-1, 1], & x_i = b. \end{cases}$$

The subgradient of $L(b)$ is the sum of these contributions. At the optimum we must have

$$0 \in \partial L(b).$$

This condition means that the number of data points strictly smaller than b and the number strictly larger than b must balance each other: neither side may contain more than half of the samples. Any value of b that satisfies this property is a median of the sample.

Thus, minimizing the ℓ_1 loss selects the sample median.

Exercise 2

We want to show that affine functions of the form

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b$$

are equivalent to linear functions on the augmented vector $(\mathbf{x}, 1)$.

Define the augmented input and weight vector

$$\tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} \in \mathbb{R}^{d+1}, \quad \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} \in \mathbb{R}^{d+1}.$$

Then

$$\tilde{\mathbf{x}}^\top \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{x}^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} = \mathbf{x}^\top \mathbf{w} + b = f(\mathbf{x}).$$

Thus every affine function $\mathbf{x}^\top \mathbf{w} + b$ can be written as a linear function $\tilde{\mathbf{x}}^\top \tilde{\mathbf{w}}$ on the augmented input $(\mathbf{x}, 1)$. Conversely, any linear function on $(\mathbf{x}, 1)$ has the form $\tilde{\mathbf{x}}^\top \tilde{\mathbf{w}}$ and therefore corresponds to an affine function in $\mathbf{x}$. Hence the two classes of functions are equivalent.